

ROBOSENSE LIDAR MODEL: CHANANGLES

Saturday, September 12, 2026 9:39 PM

purpose of LIDAR: mapping + localization w/ depth measurem. of env.

take LIDAR measurements + $\hat{p}$ = [surface type, weather, inclination $\angle$] =>

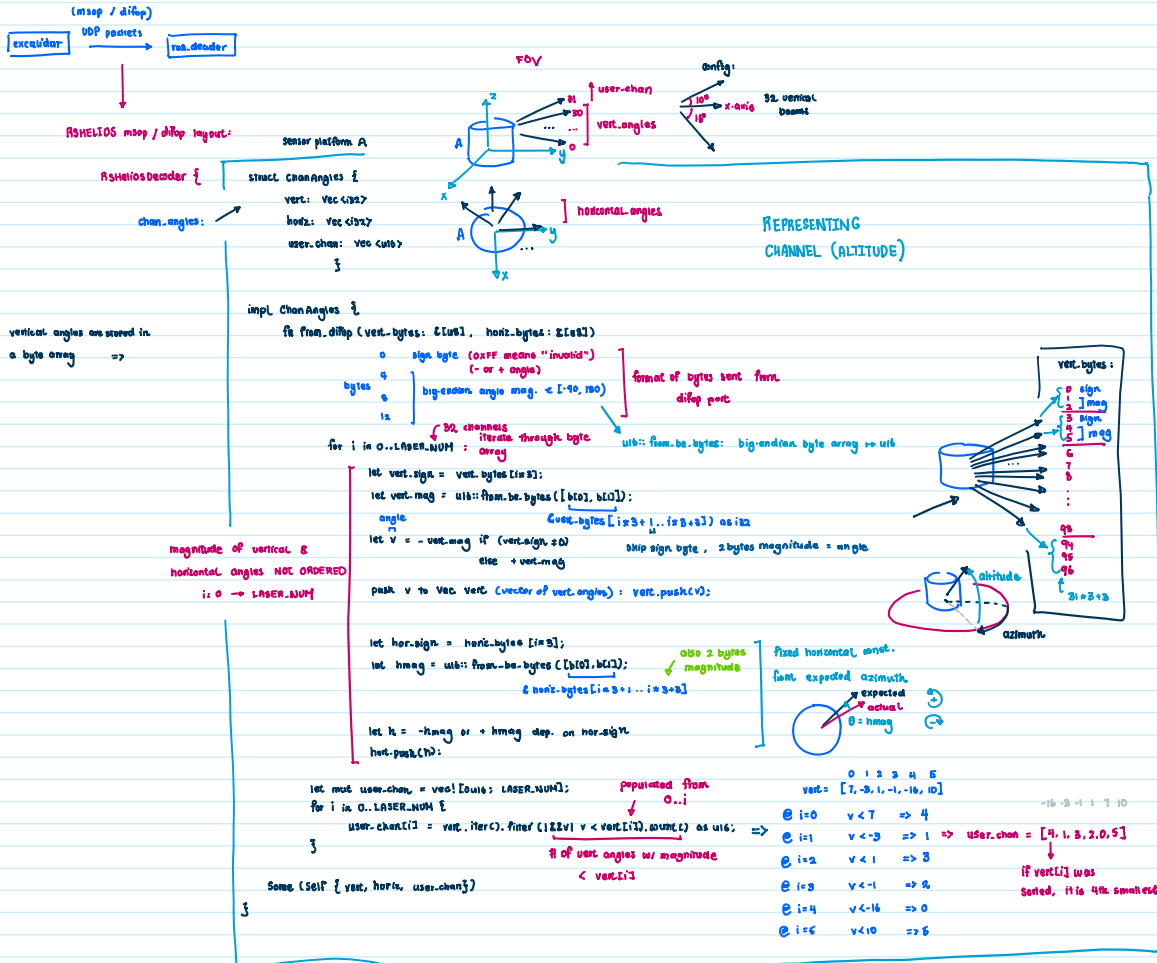
probabilistic model of noise

z-buffering - ray-casting to compute depth

compute what is occluded or not

=> shadow-mapping

ray-tracing to account for transmitting materials



lidar data packet contents:
x, y, z, reflection intensity, timestamp

32 channels
0.5 m blind zone
360° horizontal FOV, vertical FOV: 25° -> -15° to +10°
0.5° horizontal angle resolution => 720 hor. channels
20 Hz frame rate (1000 rpm)
laser emission angle: hor: 3mrad
vert: 5mrad

AZIMUTHSECTION

Saturday, September 12, 2026 11:48 PM

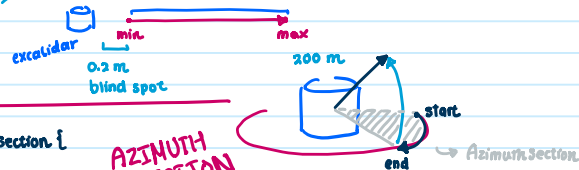
block-az-diff-const: 132,
fov-blind-ts-diff:

struct DistanceSection { min max }

distance_section: DistanceSection

=> DistanceSection::new(cfg.min_distance, cfg.max_distance)

impl DistanceSection {
fn new(user_min, user_max) { }



scan_section: AzimuthSection

struct AzimuthSection {
full-round
start: i32
end: i32
cross-zero
}

AZIMUTH SECTION

azimuth_round(v: i32) -> {
((v % 36000) + 36000) % 36000
} in centidegrees, round deg

impl AzimuthSection {

fn new(start, end) -> Self {

init start, end

section is either 0° or 360° sweep.

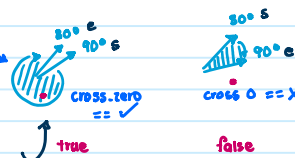
let full_round = azimuth_round(end - start) == 0;

cross-zero: start > end

azimuth_rounded

fn contains:

whether angle in section or not



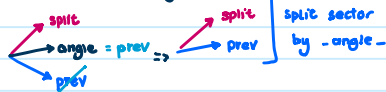
split: SplitStrategyByAngle =>

struct SplitStrategyByAngle {
split_angle prev_angle
}

impl SplitStrategyByAngle {

fn new(split_angle) { self { split_angle =
prev_angle: split_angle } }

fn new_block(angle) {



points: Vec<Point>

first_point_ts:

prev_point_ts:

chan_tx: [f64; 2]

chan_rx: [f64; 2]

Sender Receiver

let tx, rx = mpsc::channel::<RawPacket>();

main() -> run_decoder(config_path) -> spawn_packet_source()

Packet Processing stuff

live: spawn_udp_reader(): bind msoep / difop ports
offline: spawn_pcap_reader()

-> RSHeliosDecoder::new(cfg): constructor

run_decoder:

read each packet
receiver

get RPM, RPS, forecast

these info in 32-34 of packet

GOAL IS TO GET DEPTH +
CORR. ANGLES TO GET
XYZ

